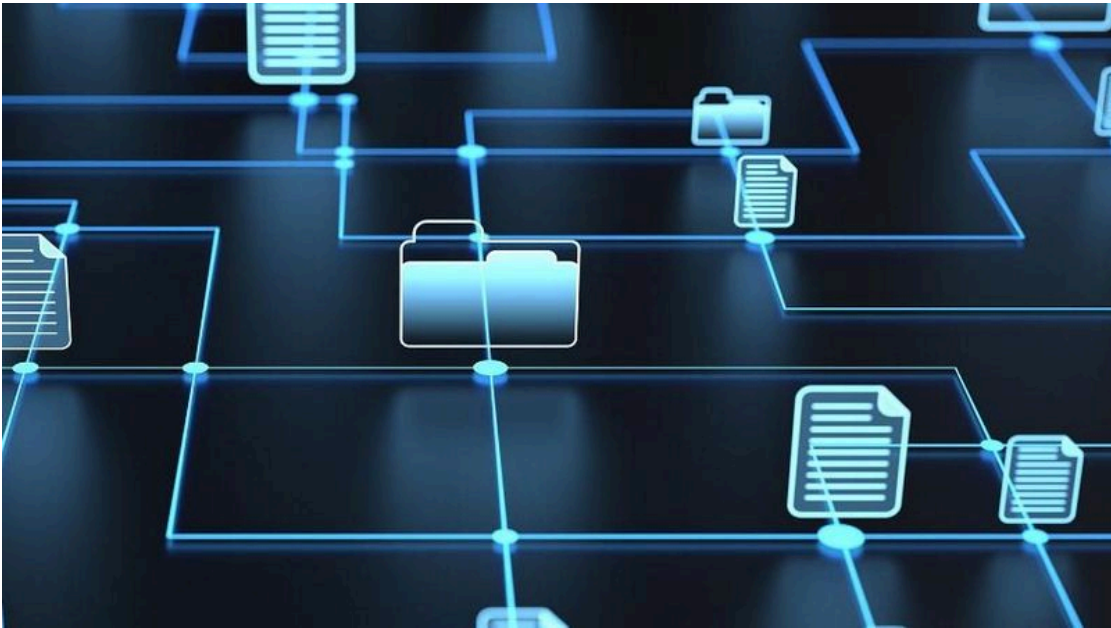


Base de Dados

LEI - Licenciatura de Engenharia Informática

2025/26 - 1º semestre

SIESTÁGIOS - 2ª parte



Membros do Grupo 69:

David Chagas Vicente, nº 129875

João Francisco Branco de Almeida, nº 129862

Martim Pereira de Barros, nº 129873

Pedro Miguel Nunes Coelho, nº 129825

Docente:

José Rafael Moita Coelho (**Turno:** 03724PL04)

0. Introdução	3
1. Automatismos	4
1.1. Triggers	4
T1: Validar a Classificação do Estagiário (1 a 5)	4
T2: Validar Consistência de Datas do Estágio	5
1.2. Stored Procedures	6
P1: Registrar um Estágio com Validações	6
P2: Listar Estágios a Iniciar em X Dias	8
1.3. Funções	10
F1: Média das Classificações de um Estabelecimento	10
F2: Média Ponderada da Nota Final	11
2. Pesquisa de Dados	13
QP - Questões de pesquisa	13
Q1: Formadores e total de estágios supervisionados	13
Q2: Empresas com média de notas superior a 14	13
Q3: Empresas e total de produtos	14
Q4: Empresas e total de estágios	14
Q5: Cursos com turmas acima da média	15
Q6: Formadores "acima da média"	15
VD - Vista de Detalhe	16
V1: Detalhes dos estágios por formador	16
V2: Médias por Empresa e Curso	16
3. Sistema Web-Based (Protótipo)	18
3.1. Ligação e Autenticação	18
3.2. Portal do Administrador (W1)	19
3.3. Portal do Aluno (W2)	19
3.4. Portal do Formador (W3)	20
4. Conclusão	22
4.1. Síntese do Trabalho Desenvolvido	22
4.2. Dificuldades e Críticas	22
4.3. Apreciação Final	22

0. Introdução

O presente documento constitui a segunda parte do projeto da disciplina de Bases de Dados, **focado na operacionalização e interatividade do sistema "SIESTÁGIOS"**. A principal meta desta fase foi sair do modelo conceptual, **implementando mecanismos “mais complexos” de bases de dados (automatismos e consultas)** e, posteriormente, **desenvolvendo um protótipo de sistema *Web-Based* para a gestão e interação com os dados.**

Esta etapa abrange a criação de *Triggers*, *Stored Procedures* e *Functions* para garantir a integridade e a automatização da lógica de negócio. Inclui também o desenvolvimento de *Views* para suportar a pesquisa e análise de dados. Finalmente, estes elementos são integrados num sistema *web* funcional, com perfis distintos (Administrador, Aluno e Formador), demonstrando a aplicação prática e a comunicação entre a *interface* e o motor de base de dados. O presente documento constitui a segunda parte do projeto da disciplina de Bases de Dados, **focado na operacionalização e interatividade do sistema "SIESTÁGIOS"**. Esta fase crítica do projeto marca a transição do **modelo conceptual para uma implementação prática e funcional**. A principal meta foi ir além da representação estática dos dados, **implementando mecanismos “mais complexos” de bases de dados (automatismos e consultas)** e, posteriormente, **desenvolvendo um protótipo de sistema *Web-Based* para a gestão e interação com os dados.**

Esta etapa abrange, de forma detalhada, a criação de *Triggers*, *Stored Procedures* e *Functions* para garantir a integridade, a segurança e a automatização da lógica de negócio complexa associada à gestão de estágios. Inclui também o desenvolvimento de *Views* estratégicas para suportar a pesquisa eficiente, relatórios e a análise de dados por diferentes perfis de utilizadores.

Finalmente, o culminar deste trabalho é a integração destes elementos num sistema *web* funcional, com perfis distintos (Administrador, Aluno e Formador), demonstrando a aplicação prática e robusta dos conhecimentos adquiridos e a comunicação eficaz entre a *interface* de utilizador e o motor de base de dados.

1. Automatismos

1.1. Triggers

T1: Validar a Classificação do Estagiário (1 a 5)

Código 1 SQL - Trigger 1

```
DELIMITER $$

-- Trigger para INSERÇÃO
CREATE TRIGGER `T1_insert` BEFORE INSERT ON `estagio` FOR EACH ROW BEGIN
    -- Verifica se a classificação é inferior a 1 ou superior a 5
    IF NEW.classificacao < 1 OR NEW.classificacao > 5 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro T1: A classificação atribuída deve ser entre 1 e 5.';
    END IF;
END$$

-- Trigger para ATUALIZAÇÃO
CREATE TRIGGER `T1_update` BEFORE UPDATE ON `estagio` FOR EACH ROW BEGIN
    -- Verifica se a classificação é inferior a 1 ou superior a 5
    IF NEW.classificacao < 1 OR NEW.classificacao > 5 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro T1: A classificação atribuída deve ser entre 1 e 5.';
    END IF;
END$$

DELIMITER ;
```

Explicação:

- Foram criados dois triggers (BEFORE INSERT e BEFORE UPDATE) na tabela estágio.
- O objetivo é garantir a integridade dos dados na coluna classificação.

- O código verifica se o valor novo (NEW.classificacao) é menor que 1 ou maior que 5.
- Caso a condição se verifique, o trigger interrompe a operação e lança um erro personalizado (SIGNAL SQLSTATE '45000') com uma mensagem explicativa.

T2: Validar Consistência de Datas do Estágio

Código SQL - Trigger 2

```
DELIMITER $$

CREATE TRIGGER `T2` BEFORE UPDATE ON `estagio` FOR EACH ROW BEGIN
    -- Verifica se a data de início é posterior à data de fim
    IF NEW.data_inicio > NEW.data_fim THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro T2: A data de início não pode ser posterior à data de fim.';
    END IF;
END$$

DELIMITER ;
```

Explicação:

- Este trigger atua antes de uma atualização (BEFORE UPDATE) na tabela estagio.
- Compara a nova data de início (NEW.data_inicio) com a nova data de fim (NEW.data_fim).
- Se a data de início for cronologicamente posterior à data de fim, o sistema impede a gravação e devolve uma mensagem de erro, garantindo a consistência temporal dos estágios.

1.2. Stored Procedures

P1: Registrar um Estágio com Validações

Código SQL - SP 1

```
DELIMITER $$

CREATE PROCEDURE `P1` (IN `p_data_inicio` DATE, IN `p_data_fim` DATE, IN `p_aluno_id`
INT, IN `p_empresa_id` INT, IN `p_estabelecimento_id` INT, IN `p_formador_id` INT) BEGIN
    -- Variáveis para contagem
    DECLARE v_aluno_exists INT;
    DECLARE v_estab_exists INT;
    DECLARE v_formador_exists INT;

    -- 1. Verificar se o ALUNO existe (PK: utilizador_id)
    SELECT COUNT(*) INTO v_aluno_exists
    FROM aluno
    WHERE utilizador_id = p_aluno_id;

    -- 2. Verificar se o ESTABELECIMENTO existe (PK composta: empresa_id +
estabelecimento_id)
    SELECT COUNT(*) INTO v_estab_exists
    FROM estabelecimento
    WHERE empresa_id = p_empresa_id AND estabelecimento_id = p_estabelecimento_id;

    -- 3. Verificar se o FORMADOR existe (PK: utilizador_id)
    SELECT COUNT(*) INTO v_formador_exists
    FROM formador
    WHERE utilizador_id = p_formador_id;

    -- Validação: Se algum não existir, lança erro
    IF v_aluno_exists = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro P1: O Aluno indicado não
existe.';
    ELSEIF v_estab_exists = 0 THEN
```

```

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro P1: O Estabelecimento indicado
não existe.';
    ELSEIF v_formador_exists = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Erro P1: O Formador indicado não
existe.';
    ELSE
        -- Se todos existirem, insere o estágio
        INSERT INTO estagio (
            data_inicio, data_fim, aluno_id,
            estabelecimento_empresa_id, estabelecimento_id, formador_id
        )
        VALUES (
            p_data_inicio, p_data_fim, p_aluno_id,
            p_empresa_id, p_estabelecimento_id, p_formador_id
        );
    END IF;
END$$

DELIMITER ;

```

Explicação:

- Este procedimento recebe como parâmetros os dados necessários para criar um estágio.
- Antes de inserir, verifica a integridade referencial manualmente: conta (COUNT) se o Aluno, o Estabelecimento e o Formador existem nas respectivas tabelas.
- Se algum destes contadores for 0 (não existe), o procedimento aborta e lança um erro específico indicando qual a entidade em falta.
- Se todas as validações passarem, execute o comando INSERT na tabela de estágio.

P2: Listar Estágios a Iniciar em X Dias

Código SQL - SP 2

```

DELIMITER $$

CREATE PROCEDURE `P2` (IN `p_dias` INT) BEGIN
    SELECT

```

```

    e.data_inicio,
    e.data_fim,
    u.nome AS nome_aluno,
    est.nome_comercial AS local_estagio,
    est.localidade
FROM
    estagio
-- Ligar ao Aluno e depois ao Utilizador para buscar o nome
INNER JOIN aluno a ON e.aluno_id = a.utilizador_id
INNER JOIN utilizador u ON a.utilizador_id = u.utilizador_id
-- Ligar ao Estabelecimento (chave composta)
INNER JOIN estabelecimento est ON
    e.estabelecimento_id = est.estabelecimento_id AND
    e.estabelecimento_empresa_id = est.empresa_id
WHERE
    -- A data de início tem de ser hoje ou futura
    e.data_inicio >= CURDATE()
    AND
    -- E tem de ser menor ou igual à data de hoje + X dias
    e.data_inicio <= DATE_ADD(CURDATE(), INTERVAL p_dias DAY);
END$$

DELIMITER ;

```

Explicação:

- Este procedimento aceita um número de dias (p_dias) como parâmetro.
- Realiza uma consulta (SELECT) que junta as tabelas de estágio, aluno (e utilizador para obter o nome) e estabelecimento.
- A cláusula WHERE filtra os resultados usando funções de data do MySQL:
 1. CURDATE(): Garante que o estágio começa hoje ou no futuro.
 2. DATE_ADD(...): Define o limite superior do intervalo, somando o parâmetro p_dias à data atual.

1.3. Funções

F1: Média das Classificações de um Estabelecimento

Código SQL - F1

```
DELIMITER $$

CREATE FUNCTION `F1` (`p_empresa_id` INT, `p_estabelecimento_id` INT, `p_ano_letivo`
VARCHAR(150)) RETURNS DOUBLE DETERMINISTIC READS SQL DATA BEGIN
    DECLARE v_media DOUBLE;

    -- Calcula a média
    SELECT AVG(media) INTO v_media
    FROM classificacao
    WHERE
        estabelecimento_empresa_id = p_empresa_id
        AND
        estabelecimento_id = p_estabelecimento_id
        AND
        ano_letivo = p_ano_letivo;

    -- Se não encontrar nada (NULL), devolve 0
    RETURN IFNULL(v_media, 0);
END$$

DELIMITER ;
```

Explicação:

- A função recebe o ID da empresa, o ID do estabelecimento e o ano letivo.
- Utiliza a função de agregação AVG() na tabela classificacao para calcular a média das classificações atribuídas àquele estabelecimento específico naquele ano.
- Utiliza IFNULL no retorno para garantir que, se não houverem classificações registradas (o que resultaria em NULL), a função devolve 0 em vez de um valor vazio.

F2: Média Ponderada da Nota Final

Código SQL - F2

```
DELIMITER $$

CREATE FUNCTION `F2` (`p_aluno_id` INT, `p_data_inicio` DATE, `p_peso_empresa`
DECIMAL(5,2), `p_peso_escola` DECIMAL(5,2), `p_peso_relatorio` DECIMAL(5,2),
`p_peso_procura` DECIMAL(5,2)) RETURNS DECIMAL(10,2) DETERMINISTIC READS SQL
DATA BEGIN

    -- Variáveis para guardar as notas vindas da tabela
    DECLARE v_n_empresa DOUBLE;
    DECLARE v_n_escola DOUBLE;
    DECLARE v_n_relatorio DOUBLE;
    DECLARE v_n_procura DOUBLE;

    DECLARE v_media_final DECIMAL(10,2);
    DECLARE v_soma_pesos DECIMAL(5,2);

    -- 1. Buscar as notas à tabela ESTAGIO
    SELECT
        IFNULL(nota_empresa, 0),
        IFNULL(nota_escola, 0),
        IFNULL(nota_relatorio, 0),
        IFNULL(nota_procura, 0)
    INTO
        v_n_empresa, v_n_escola, v_n_relatorio, v_n_procura
    FROM estagio
    WHERE
        aluno_id = p_aluno_id
        AND data_inicio = p_data_inicio;

    -- 2. Calcular a soma dos pesos para a divisão
    SET v_soma_pesos = p_peso_empresa + p_peso_escola + p_peso_relatorio + p_peso_procura;

    -- Proteção contra divisão por zero
    IF v_soma_pesos = 0 THEN
```

```

    RETURN 0;
END IF;

-- 3. Cálculo da Média Ponderada
SET v_media_final = (
    (v_n_empresa * p_peso_empresa) +
    (v_n_escola * p_peso_escola) +
    (v_n_relatorio * p_peso_relatorio) +
    (v_n_procura * p_peso_procura)
) / v_soma_pesos;

RETURN v_media_final;
END$$

DELIMITER ;

```

Explicação:

- Esta função calcula a nota final de um estágio específico (identificado por aluno e data de início).
- Recebe como parâmetros os 4 pesos diferentes para cada componente da avaliação.
- Primeiro, faz um SELECT para guardar as notas parciais do estágio em variáveis locais, tratando valores nulos como 0.
- Calcula a soma dos pesos para evitar erros matemáticos (divisão por zero).
- Aplica a fórmula da média ponderada: $(Nota1 * Peso1 + \dots + NotaN * PesoN) / SomaPesos$ e devolve o resultado arredondado.

2. Pesquisa de Dados

Nesta secção apresentam-se as Views criadas para responder às questões de pesquisa (Q1 a Q6) e as vistas de detalhe (V1 e V2).

QP - Questões de pesquisa

Q1: Formadores e total de estágios supervisionados

Listar formadores com mais de 1 estágio.

Código SQL - Q1

```
CREATE VIEW Q1 AS
SELECT
    u.nome AS Nome_Formador,
    COUNT(e.formador_id) AS Total_Estagios
FROM formador f
INNER JOIN utilizador u ON f.utilizador_id = u.utilizador_id
INNER JOIN estagio e ON f.utilizador_id = e.formador_id
GROUP BY f.utilizador_id, u.nome
HAVING COUNT(e.formador_id) > 1;
```

Explicação: Junta as tabelas formador, utilizador e estagio. Agrupa pelo ID do formador e filtra (HAVING) apenas aqueles cuja contagem de estágios associados seja superior a 1.

Q2: Empresas com média de notas superior a 14

Listar empresas cuja média das notas atribuídas aos estagiários é ≥ 14 .

Código SQL - Q2

```
CREATE VIEW Q2 AS
SELECT
    emp.firma AS Nome_Empresa,
    FORMAT(AVG(est.nota_empresa), 2) AS Media_Notas
FROM empresa emp
INNER JOIN estagio est ON emp.empresa_id = est.estabelecimento_empresa_id
```

```
GROUP BY emp.empresa_id, emp.firma  
HAVING AVG(est.nota_empresa) >= 14;
```

Explicação: Calcula a média da coluna nota_empresa da tabela estagio, agrupada por empresa. A função FORMAT(..., 2) arredonda o resultado a duas casas decimais.

Q3: Empresas e total de produtos

Listar empresas que comercializam pelo menos 1 produto.

Código SQL - Q3

```
CREATE VIEW Q3 AS  
SELECT  
    e.firma AS Nome_Empresa,  
    COUNT(c.produto_id) AS Total_Produtos  
FROM empresa e  
INNER JOIN comercializa c ON e.empresa_id = c.estabelecimento_empresa_id  
GROUP BY e.empresa_id, e.firma  
HAVING COUNT(c.produto_id) >= 1;
```

Explicação: Realiza a contagem de registros na tabela de associação comercializa para cada empresa. O INNER JOIN garante que apenas empresas com correspondência na tabela comercializa são consideradas.

Q4: Empresas e total de estágios

Listar empresas com pelo menos 1 estágio.

Código SQL - Q4

```
CREATE VIEW Q4 AS  
SELECT  
    e.firma AS Nome_Empresa,  
    COUNT(est.aluno_id) AS Total_Estagios  
FROM empresa e  
INNER JOIN estagio est ON e.empresa_id = est.estabelecimento_empresa_id  
GROUP BY e.empresa_id, e.firma  
HAVING COUNT(est.aluno_id) >= 1;
```

Explicação: Semelhante à anterior, mas conta o número de ocorrências na tabela estagio associadas a cada empresa.

Q5: Cursos com turmas acima da média

Cursos cujo n° de turmas é superior à média geral de turmas por curso.

Código SQL - Q5

```
CREATE VIEW Q5 AS
SELECT
    c.designacao AS Nome_Curso,
    COUNT(t.turma_id) AS Total_Turmas
FROM curso c
INNER JOIN turma t ON c.curso_id = t.curso_id
GROUP BY c.curso_id, c.designacao
HAVING COUNT(t.turma_id) > (
    SELECT COUNT(*) FROM turma) / (SELECT COUNT(*) FROM curso
);
```

Explicação: Utiliza uma subquery no HAVING para calcular a média global (Total de Turmas / Total de Cursos). Compara depois a contagem de turmas de cada curso individual com esse valor global.

Q6: Formadores "acima da média"

Formadores cuja média de notas é superior à média global.

Código SQL - Q6

```
CREATE VIEW Q6 AS
SELECT
    u.nome AS Nome_Formador,
    FORMAT(AVG(e.nota_final), 2) AS Media_Formador,
    FORMAT((SELECT AVG(nota_final) FROM estagio), 2) AS Media_Global
FROM formador f
INNER JOIN utilizador u ON f.utilizador_id = u.utilizador_id
INNER JOIN estagio e ON f.utilizador_id = e.formador_id
GROUP BY f.utilizador_id, u.nome
HAVING AVG(e.nota_final) > (SELECT AVG(nota_final) FROM estagio);
```

Explicação: Calcula a média das notas finais dos estágios de cada formador e compara-a com uma subquery que devolve a média de todos os estágios registados na base de dados.

VD - Vista de Detalhe

V1: Detalhes dos estágios por formador

Vista detalhada com totais e médias por formador.

Código SQL - V1

```
CREATE VIEW V1 AS
SELECT
    u.nome AS Nome_Formador,
    COUNT(e.aluno_id) AS Num_Estagios,
    FORMAT(AVG(e.nota_final), 2) AS Media_Formador,
    FORMAT((SELECT AVG(nota_final) FROM estagio), 2) AS Media_Geral
FROM formador f
INNER JOIN utilizador u ON f.utilizador_id = u.utilizador_id
INNER JOIN estagio e ON f.utilizador_id = e.formador_id
GROUP BY f.utilizador_id, u.nome;
```

Explicação: Agrega informação estatística (contagem e média) por formador, fornecendo também a média global para contexto comparativo.

V2: Médias por Empresa e Curso

Vista agrupada por empresa e curso.

Código SQL - V2

```
CREATE VIEW V2 AS
SELECT
    emp.firma AS Nome_Empresa,
    c.designacao AS Nome_Curso,
    FORMAT(AVG(e.nota_final), 2) AS Media_Notas
FROM empresa emp
INNER JOIN estagio e ON emp.empresa_id = e.estabelecimento_empresa_id
```

```
INNER JOIN aluno a ON e.aluno_id = a.utilizador_id  
INNER JOIN turma t ON a.turma_id = t.turma_id  
INNER JOIN curso c ON t.curso_id = c.curso_id  
GROUP BY emp.empresa_id, emp.firma, c.curso_id, c.designacao  
ORDER BY emp.firma ASC, c.designacao ASC;
```

Explicação: Realiza múltiplos JOINS para ligar a Empresa ao Curso (Empresa -> Estágio -> Aluno -> Turma -> Curso), permitindo calcular a média das notas finais segmentada por estas duas dimensões.

3. Sistema Web-Based (Protótipo)

O sistema foi desenvolvido utilizando a arquitetura Cliente-Servidor, recorrendo à linguagem **PHP** para o processamento do lado do servidor e **MySQL (MariaDB)** para a persistência de dados. A conexão à base de dados foi implementada através da biblioteca `mysqli`, garantindo compatibilidade e segurança.

A estrutura do projeto foi organizada em pastas distintas (admin, aluno, formador) para separar os perfis de acesso, controlados via variáveis de sessão (`$_SESSION`).

3.1. Ligação e Autenticação

Foi criado um ficheiro central (`db.php`) para gerir a conexão, incluído em todas as páginas. O login valida as credenciais e redireciona o utilizador consoante o seu tipo (campo tipo na tabela utilizador).

Excerto do Login (`index.php`):

PHP

```
// Validação de credenciais e criação de sessão
```

```
$sql = "SELECT utilizador_id, nome, tipo FROM utilizador WHERE login='$login' AND password='$pass'";
```

```
$result = mysqli_query($conn, $sql);
```

```
if (mysqli_num_rows($result) == 1) {
```

```
    $row = mysqli_fetch_assoc($result);
```

```
    $_SESSION['user_id'] = $row['utilizador_id'];
```

```
    $_SESSION['tipo'] = $row['tipo']; // Define o perfil (admin, aluno, formador)
```

```
// Redirecionamento dinâmico
```

```
if($row['tipo'] == 'administrativo') header("Location: admin/dashboard.php");
```

```
elseif($row['tipo'] == 'aluno') header("Location: aluno/dashboard.php");
```

```
elseif($row['tipo'] == 'formador') header("Location: formador/dashboard.php");
```

```
}
```

3.2. Portal do Administrador (W1)

Este portal permite a gestão integral dos estágios e alunos.

Registo de Estágios (Uso de Stored Procedures) Para cumprir o requisito de validação de existências (Aluno, Empresa, Formador), o PHP não insere diretamente na tabela, mas sim invoca o Stored Procedure P1 desenvolvido na fase de Automatismos.

Código utilizado (admin/inserir_estagio.php):

PHP

```
// O PHP invoca o procedimento P1 passando os dados do formulário
$sql = "CALL P1('$d_ini', '$d_fim', $aluno, $emp, $estab, $form)";

if(mysqli_query($conn, $sql)) {
    echo "Estágio registado com sucesso (Validações de P1 aprovadas).";
} else {
    echo "Erro: " . mysqli_error($conn); // Retorna erro se o aluno/empresa não existir
}
```

Registo de Alunos A inserção é feita em duas etapas para garantir a integridade referencial: primeiro cria-se o utilizador e recupera-se o ID gerado (mysqli_insert_id), inserindo de seguida na tabela aluno.

3.3. Portal do Aluno (W2)

Permite a consulta de empresas e detalhes específicos dos estágios.

Filtragem de Empresas A listagem permite filtrar resultados por localidade através de parâmetros GET na URL, construindo a query SQL dinamicamente.

Código utilizado (aluno/dashboard.php):

PHP

```
$sql = "SELECT * FROM empresa WHERE 1=1";

// Se o utilizador preencheu o filtro de localidade
```

```
if(isset($_GET['local']) && !empty($_GET['local'])) {  
    $local = mysqli_real_escape_string($conn, $_GET['local']);  
    $sql .= " AND localidade LIKE '%$local%'";  
}  
  
$result = mysqli_query($conn, $sql);
```

Detalhes do Estágio (JOINS) Para exibir toda a informação requerida (Responsável, Transportes, etc.), foi utilizada uma query complexa com múltiplos JOIN.

Código utilizado (aluno/detalhes_empresa.php):

PHP

```
$sql = "SELECT est.nome_comercial, resp.nome, resp.email, t.meio_transporte  
FROM estabelecimento est  
JOIN responsavel resp ON est.responsavel_id = resp.responsavel_id  
LEFT JOIN servico s ON est.estabelecimento_id = s.estabelecimento_id  
LEFT JOIN transporte t ON s.transporte_id = t.transporte_id  
WHERE est.empresa_id = $empresa_id";
```

3.4. Portal do Formador (W3)

O formador pode lançar as notas parcelares. O sistema realiza o cálculo automático da média final antes de gravar na base de dados, cumprindo o requisito de automação.

Cálculo e Finalização (formador/avaliar.php):

PHP

```
// Recolha das notas do formulário  
$n1 = $_POST['nota_empresa'];  
$n2 = $_POST['nota_escola'];  
$n3 = $_POST['nota_procura'];  
$n4 = $_POST['nota_relatorio'];  
  
// Cálculo da média final no servidor  
$final = ($n1 + $n2 + $n3 + $n4) / 4;
```

// Atualização do registo na Base de Dados

```
$sql = "UPDATE estagio
```

```
    SET nota_empresa=$n1, nota_escola=$n2, nota_procura=$n3,
```

```
        nota_relatorio=$n4, nota_final=$final
```

```
    WHERE aluno_id=$aluno_id AND formador_id=$formador_id";
```

```
mysqli_query($conn, $sql);
```

4. Conclusão

4.1. Síntese do Trabalho Desenvolvido

Em Bases de Dados, a 2ª etapa do projeto 'SIESTÁGIOS' consistiu em **dar interatividade** ao modelo de informação anteriormente realizado, focando-se na sua **operacionalização, automatização e na criação de uma interface de interação**.

O trabalho desenvolvido estruturou-se em três pilares essenciais:

- **Automatização e Integridade da Base de Dados (SQL):** Procedeu-se à implementação de mecanismos para garantir a consistência e a validade dos dados. Foram criados **Triggers** para assegurar a conformidade de dados críticos (p.e classificações e datas) e desenvolvidas **Stored Procedures** para encapsular e simplificar a execução de lógica de negócio complexa, como o registo de validação de novos estágios. Adicionalmente, foram introduzidas **Functions** dedicadas a cálculos frequentes, como o apuramento de médias ponderadas.
- **Análise e Suporte à Decisão:** Para transformar os dados em informação útil, foram concebidas **Views**. Estas vistas respondem a indicadores de gestão predefinidos, proporcionando uma análise aprofundada sobre o desempenho de Formadores, Empresas e Cursos.
- **Desenvolvimento da Interface de Utilizador:** O objetivo final do projeto envolveu a construção de um **protótipo funcional** com arquitetura Cliente-Servidor. Esta interface web permite a interação em tempo real com a base de dados, suportando as necessidades dos três perfis distintos de utilizadores: **Administrador, Aluno e Formador**.

4.2. Dificuldades e Críticas

Durante o desenvolvimento, a principal dificuldade técnica incidiu na configuração e estabilidade do ambiente de desenvolvimento local, p.e na gestão de conflitos de portas e recuperação de dados após falhas inesperadas do serviço MySQL. Ao nível da implementação, o desafio maior foi garantir a correta comunicação entre o PHP e os automatismos criados em SQL.

4.3. Apreciação Final

Este projeto permitiu consolidar a importância de separar a lógica de dados da lógica de apresentação. A utilização de *Stored Procedures* e *Triggers* provou ser uma estratégia eficaz para garantir a integridade dos dados diretamente no motor de base de dados, impedindo que erros de programação na aplicação Web comprometam a informação.